1-Port Platform Mediated Avoidance (PMA) Behavioral Analysis Pipeline

Overview

This pipeline processes behavioral data from variable conflict 1-Port Platform Mediated Avoidance (PMA) experiments recorded in **AnyMaze**. It computes cue-aligned time-on-platform, cue-aligned nosepoke probability, latency to platform, shock outcome classification (Avoided / Escaped / Fully Shocked), total reward consumption, reward delivery during light trials, and the Reward-Avoidance Index — within sessions and across multiple session days — then produces publication-ready SVG figures and summary CSVs.

You only need to edit one file: `config.py`**.** Everything else runs automatically from `runner.py`.

Acknowledgements

This pipeline was built on top of the original analysis notebooks developed by **Hector Yarur Castillo**. His notebooks established the core logic for nosepoke histogram analysis, reward quantification, rewards-in-light-trial gating, the Reward-Avoidance Index, and shock avoidance classification that formed the foundation of this codebase. The present pipeline refactors and extends his work into a modular, automated, multi-session framework.

Table of Contents

---

1. Quick Start

1. Place all pipeline files in the same folder:

```
config.py
runner.py
utils.py
Time_on_Platform.py
Nosepokes.py
Rewards.py
PlatformLatency.py
Variable_Conflict_Shocks.py
ACROSS_SESSIONS_AUC.py
```

2. Open `config.py` in Spyder (or any editor).
3. Set `DATA_ROOT` to the full path of your `BehaviorData` folder.
4. Set your treatment group names and colors.
5. Toggle which analyses you want using `True` / `False` flags.
6. Open `runner.py` and press play (▶). All outputs are written inside `BehaviorData/Analysis/`.

---

2. File Overview

File	What it does
`config.py`	**Start here.** All user settings: paths, flags, colors, timing, and reward parameters. Do not edit anything else.
`runner.py`	Entry point. Reads config, loops over sessions, calls each analysis module.
`utils.py`	Shared functions: metadata loading, resampling, cue detection, and the core cue-aligned analysis engine. Do not edit unless you know what you are changing.
`Time_on_Platform.py`	Cue-aligned time-on-platform traces and per-mouse light-window AUC computation.

File	What it does
`Nosepokes.py`	Cue-aligned nosepoke probability traces, per-mouse light-window AUC, and full-session binned nosepoke histogram.
`Rewards.py`	Total reward consumption (mL), reward delivery during light trials, and Reward-Avoidance Index per cue type.
`PlatformLatency.py`	Within-session and across-session latency-to-platform analysis.
`Variable_Conflict_Shocks.py`	Classifies every tone-containing trial as Avoided, Escaped, or Fully Shocked. Produces within-session and across-session summary plots.
`ACROSS_SESSIONS_AUC.py`	Collects per-mouse nosepoke AUC CSVs from all sessions and produces longitudinal group plots.

3. Required Data Structure

3.1 Top-level layout

```
BehaviorData/
├── animals_metadata.xlsx           ← required (see Section 4)
├── REW01/                          ← reward session folder
|   ├── 06-08-25_10b_REW01.csv
|   ├── 06-08-25_11a_REW01.csv
|   └── downsampled/                ← created automatically if RUN_DOWNSAMPLE =
True
|       ├── 06-08-25_10b_REW01_downsampled.csv
|       └── ...
├── REW02/
├── VC01/                           ← variable conflict session folder
|   ├── 07-15-25_10b_VC01.csv
|   └── ...
├── VC02/
└── Analysis/                       ← all outputs written here (created
automatically)
```

The pipeline processes any subfolder of `BehaviorData` whose name starts with `REW` or `VC`, in alphabetical order. Set `SESSION_FILTER = "VC03"` in `config.py` to process only one session.

3.2 Session folder naming

Session folders must begin with `REW` or `VC`, followed by a two-digit index:

```
REW01   REW02   ...   VC01   VC02   ...
```

The numeric suffix (e.g. `01`, `02`) is used to sort sessions chronologically in across-session plots.

3.3 Raw CSV file naming

Each animal's AnyMaze export file must follow this pattern:

```
MM-DD-YY_BehaviorID_SessionLabel.csv
```

Examples:

```
07-15-25_10b_VC01.csv
07-15-25_11a_VC01.csv
07-15-25_4B_VC01.csv
```

- `MM-DD-YY` — date of the recording session
- `BehaviorID` — the animal's behavior ID; must match an entry in `animals_metadata.xlsx` (see Section 6)
- `SessionLabel` — must match the containing folder name (e.g. `VC01`)

The pipeline derives the behavior ID by splitting the filename on `_` and taking the **second field** (index 1). Files ending in `_downsampled.csv` are skipped automatically.

3.4 Required columns in AnyMaze CSVs

Column names are read exactly as exported by AnyMaze. The following columns must be present:

Column name (as exported)	Used by
`Time (s)`	All modules — primary time axis
`House light active`	Cue detection — marks ITI periods
`Cue light active`	Cue detection — light cue signal
`New speaker active` *or* `Speaker Channel 1 active`	Cue detection — tone cue signal
`In platform`	Platform, Latency, Shock, and Rewards modules
`Nose poke active`	Nosepoke module
`Feeder active`	Rewards module — reward consumption and rewards-in-light analysis
`In Reward`	Rewards module — Reward-Avoidance Index only; can be absent if R-A Index not needed

The following columns are dropped silently if absent:

- `Head position X / Y`, `Tail position X / Y`

---

4. Metadata Spreadsheet

`animals_metadata.xlsx` must live **directly inside** `BehaviorData/` (one level above each session folder). It links behavior IDs found in filenames to animal-level information used for grouping and coloring.

Required columns

Column	Description
`behavior_id`	The ID used in CSV filenames (e.g. `10b`, `11a`, `4B`)
`treatment_group`	Group label (e.g. `ctrl`, `KO`) — aliases handled in `config.py`

Optional columns

Column	Description
`sex`	`M` or `F` — used for sex-stratified AUC plots if present
`cohort_id`	Carried through to outputs but not used in analysis

Column name formatting

Column names in the spreadsheet are normalized automatically (lowercased, stripped). `Behavior ID`, `behavior_id`, and `BehaviorID` are all recognized. The behavior_id values themselves are also lowercased before matching (see Section 6).

Treatment group normalization

Raw labels from the spreadsheet are normalized using `TREATMENT_ALIASES` in `config.py`:

```python
TREATMENT_ALIASES = {
    "ctrl": "ctrl", "Ctrl": "ctrl", "CTRL": "ctrl",
    "ko":   "KO",   "Cre":  "KO",
}
```

Add entries here to map any variant spelling to your canonical label. Animals whose normalized treatment matches any entry in `EXCLUDED_TREATMENTS` (default: `{"misc"}`) are silently dropped from all figures and CSVs.

---

5. Configuration — Full Reference

Open `config.py`. All user-editable settings are here.

5.1 Paths

```python
DATA_ROOT       = r"Z:\path\to\your\BehaviorData"
SESSION_FILTER  = None       # None = all sessions; "VC03" = one session only
OUTPUT_DIR_NAME = "Analysis" # output folder created inside DATA_ROOT
```

Use a raw string (the `r` prefix) to avoid backslash issues on Windows.

5.2 Analysis flags

```python
RUN_DOWNSAMPLE             = False  # resample raw CSVs to 1 Hz binary; save to down
RUN_TIME_ON_PLATFORM       = True
RUN_NOSEPOKES              = True
RUN_PLATFORM_LATENCY       = True
RUN_SHOCK_SUMMARY          = True   # VC sessions only
RUN_REWARDS                = True   # total consumption + rewards in light trials +
RUN_ACROSS_SESSION_AUC     = True   # requires RUN_NOSEPOKES to have been run first
RUN_ACROSS_SESSION_LATENCY = True   # requires RUN_PLATFORM_LATENCY to have been run
RUN_ACROSS_SESSION_SHOCKS  = True   # requires RUN_SHOCK_SUMMARY to have been run fi
```

Across-session analyses read the CSVs written by their corresponding within-session analyses. If you run them in the same execution they work seamlessly. If you run them in a separate execution, the within-session outputs must already exist in `Analysis/`.

5.3 Treatment labels and colors

```python
CONTROL_TREATMENT = "ctrl"   # placed first in legends and used as baseline in plots

TREATMENT_COLORS = {
    "ctrl": "#3B3B38",
    "KO":   "#1EB2F2",
}
TREATMENT_SEM_COLORS = {     # ribbon color for ± SEM shading
    "ctrl": "#B0B0AE",
    "KO":   "#A4DAF7",
}
SEM_ALPHA = 0.65             # opacity of SEM ribbon (0 = invisible, 1 = opaque)
```

To add a new group, add one entry to each color dictionary and add animals to `animals_metadata.xlsx`. No code changes are required in any analysis module.

5.4 Shock outcome colors

```python
```

```
OUTCOME_COLORS = {
    "Avoided":       "#368ebf",
    "Escaped":       "orange",
    "Fully_Shocked": "lightcoral",
}
```

5.5 Task timing

```python
CUE_DURATIONS = {
    "Light":      30,   # seconds
    "Tone":       30,
    "Conflict":   30,
    "Light-Tone": 45,
    "Tone-Light": 45,
}
POST_TRIAL_BUFFER = 30   # seconds of data captured after cue offset in cue-aligned

LIGHT_DURATION = 30      # duration of the light cue in all trial types
LIGHT_ONSET = {          # seconds from trial start to light onset, per trial type
    "Light":      0,
    "Conflict":   0,
    "Light-Tone": 0,
    "Tone-Light": 15,    # tone plays first for 15 s, then light turns on
}
```

These values control the time axis of every cue-aligned plot and the windows used for AUC and reward computation. Adjust them here if your protocol differs.

5.6 Cue dictionary

```python
CUE_DICT_SOURCE = "first_csv"   # "config" | "first_csv" | "per_animal"
```

`CUE_DICT_SOURCE` controls how trial onset times are determined for every analysis in the pipeline. The three options are:

`"config"` — trial onset times are read directly from the `CUE_DICT` dictionary defined below in `config.py`. The same times are applied to every animal in every session. No `downsampled/` folder is needed. This is the most transparent and reproducible option, and is recommended when your protocol uses fixed, known trial timing.

```python
CUE_DICT = {
    "Light":      [300, 600, 900, 1200, 1500, 1800],
    "Tone":       [360, 660, 960, 1260, 1560, 1860],
    "Conflict":   [420, 720, 1020, 1320, 1620, 1920],
    "Light-Tone": [],
    "Tone-Light": [],
}
```

All five keys must be present. Set unused trial types to empty lists. Times are in seconds from the start of the recording.

`"first_csv"` (default) — trial onset times are auto-detected from the first available animal's downsampled CSV in each session folder. The detected times are then shared across all animals in that session. Requires a `downsampled/` subfolder. Useful when trial timing is consistent across animals but varies between sessions, or when you do not want to look up onset times manually.

`"per_animal"` — trial onset times are auto-detected independently from each animal's own downsampled CSV. Each animal is analyzed against its own detected trial schedule. This is the appropriate choice if animals within a session ran on genuinely different trial schedules. Requires a `downsampled/` subfolder for every animal. The first animal's detected times serve as an internal fallback for any module that requires a single representative dictionary (e.g. group-level plots that need to know which cues exist in the session).

When `CUE_DICT_SOURCE` is `"first_csv"` or `"per_animal"` and no `downsampled/` folder is found, the session is skipped with a warning. Run with `RUN_DOWNSAMPLE = True` first to generate the downsampled files.

5.7 Reward parameters

```python
FEEDER_RATE_UL_PER_SEC = 16   # µL dispensed per second of Feeder active
```

This is the only number to change if you switch pump hardware. The default of 16 µL/s was calibrated for the pump used in Yarur's original notebooks. All three reward analyses (`compute_reward_consumption`, `compute_rewards_in_light_trials`, and the feeder-active counts in the R-A Index) derive their volume estimates from this single value.

---

6. How Files and Animals Are Matched

6.1 Extracting the behavior ID from a filename

Given a filename `07-15-25_10b_VC01.csv`, the pipeline splits on `_` and takes the second element:

```
["07-15-25", "10b", "VC01"]
               ↑
        behavior_id = "10b"
```

This value is then lowercased (`"10b"`) before being looked up in the metadata index.

6.2 Matching to metadata

The `behavior_id` column of `animals_metadata.xlsx` is lowercased and set as the DataFrame index. A match succeeds when the lowercased filename behavior ID is found in this index. If no match is found, the animal is assigned `treatment_group = "Unknown"` and excluded from group figures (but its data still appears in per-mouse grids).

To avoid mismatches, ensure that the `behavior_id` values in your spreadsheet exactly match the second field of your filenames, modulo case.

---

7. How Trials Are Detected

7.1 Downsampling

Before trial detection, raw AnyMaze CSVs are resampled from their native recording rate to **1 Hz** using pandas `resample("1s").mean()`. Two resampling modes are used:

- **Fractional (cue-aligned traces and reward analyses):** The resampled value is the mean of all raw samples within that second. Because the raw signal is binary, the mean equals the proportion of the second during which the behavior was active — e.g. a value of 0.7 for `In platform` means the animal was on the platform for 700 ms of that second.
- **Binary (cue detection and shock classification):** After resampling by mean, values ⩾ 0.5 are rounded to 1 and values < 0.5 are rounded to 0. This is used for trial type detection and shock outcome classification. Downsampled binary files are saved to `downsampled/` when `RUN_DOWNSAMPLE = True`.

7.2 ITI detection

The pipeline identifies inter-trial intervals (ITIs) as periods where the house light is on and no tone is playing:

```
is_ITI[t] = (House light active[t] == 1) AND (tone_cue[t] == 0)
```

The end of each ITI — the last sample satisfying both conditions before the state changes — marks the boundary between an ITI and the next trial.

7.3 Trial type classification

At each ITI-end boundary, the pipeline inspects the next 50 samples (~50 seconds at 1 Hz) of `Cue light active` and `Tone cue`. The relative timing of the first activation of each signal determines the trial type:

Detected pattern	Trial type
Tone only, no cue light	`Tone`
Cue light only, no tone	`Light`
Cue light and tone activate at the same sample	`Conflict`
Tone activates first, cue light follows	`Tone-Light`
Cue light activates first, tone follows	`Light-Tone`

"Activates first" is determined by comparing `idxmax()` of each cue's binary series over the inspection window — this returns the index of the first `True` value.

7.4 Cue dictionary resolution

At the start of each session, the pipeline resolves the cue dictionary according to `CUE_DICT_SOURCE` in `config.py`. Three modes are available:

`"config"` `CUE_DICT` from `config.py` is used directly. The same onset times are applied to every animal in every session. Auto-detection is not run and `downsampled/` files are not read for this purpose. This is the most reproducible option.

`"first_csv"` The pipeline reads the first available animal's downsampled CSV in the session's `downsampled/` folder, detects ITI boundaries (Section 7.2), and classifies trial types from the 50-sample inspection window (Section 7.3). The resulting dictionary is then shared identically across all other animals in that session. If the `downsampled/` folder does not exist or contains no parseable files, the session is skipped.

`"per_animal"` The same auto-detection procedure is run independently for every animal's own downsampled CSV. Each animal is analyzed against its own detected trial schedule, so animals within a session can have different sets of trial onset times. This is the appropriate mode when individual animals were run at different times or with different protocol variants. The first animal's detected dictionary is retained as an internal fallback for group-level operations that

require a single representative cue set (e.g. determining which cue types to include in a group-mean figure).

Regardless of mode, the resolved dictionary maps trial type labels to lists of absolute onset times in seconds:

```python
{
    "Light":      [120.0, 330.0, 540.0],
    "Tone":       [210.0, 420.0, 630.0],
    "Conflict":   [240.0, 450.0],
    "Light-Tone": [],
    "Tone-Light": [],
}
```

---

8. The Math

8.1 Resampling to 1 Hz (fractional)

For cue-aligned analyses and reward analyses, the raw AnyMaze data is resampled to 1 Hz by computing the **mean** of all samples within each 1-second bin:

```
fraction_in_second_k = mean( signal[t ∈ [k, k+1)] )
```

Because the raw signal is binary, the mean equals the proportion of the second during which the behavior was active. A value of 1.0 means the animal was continuously on the platform (or nosepoking, or in the reward zone) for the entire second; 0.5 means it was active for exactly half.

8.2 Cue-aligned trace construction

For each animal and each trial type, all individual trial traces are aligned to cue onset (`t = 0`) and averaged:

1. For each cue onset at time `t_onset`, extract the behavioral signal from `t_onset - 15 s` to `t_onset + cue_duration + 30 s`.
2. Reject any window that does not contain the exact expected number of samples (protects against recordings that start or end mid-window).
3. Stack all valid windows into a matrix of shape `(n_trials, n_timepoints)`.
4. Compute the trial-averaged mean and SEM across trials:

The per-animal mean and SEM traces are saved to the combined summary CSV.

8.3 Group mean traces and SEM ribbon

For each treatment group, per-animal mean traces are averaged across animals at each timepoint:

```
group_mean[t]  = (1 / n_animals) × Σⱼ  animal_mean_j[t]
group_SEM[t]   = SD_across_animals[t] / √n_animals
```

The shaded ribbon in group plots represents `group_mean[t] ± group_SEM[t]`. Animals with no valid trials for a given cue are excluded from that cue's group calculation.

8.4 Full-session nosepoke histogram

In addition to the cue-aligned traces, the nosepoke module produces a whole-session view of nosepoke engagement. The full session is divided into non-overlapping 30-second bins, and the total seconds of nosepoke activity within each bin is summed:

```
nosepoke_count[bin_k] = Σ  Nose poke active[t]   for t ∈ [k×30, (k+1)×30)
```

Because the signal is fractional after 1 Hz resampling, each sample contributes its fraction (rather than a discrete count), so the sum equals the total seconds spent nosepoking within that bin. Cue periods are shaded on the plot using the auto-detected cue onset times from the shared cue dictionary. One panel per animal is produced in a grid layout.

This figure is a whole-session overview, not a trial-type analysis, and is intended to help identify session-wide engagement patterns and verify that trial timing was as expected.

8.5 AUC during the light window

A per-mouse Area Under the Curve (AUC) is computed from the per-animal cue-aligned mean trace during the **light-on period only** of each light-containing trial type (`Light`, `Conflict`, `Light-Tone`, `Tone-Light`).

Step 1 — Baseline: Compute the mean behavioral fraction over the 10 seconds immediately preceding light onset:

```
baseline_mean = mean( signal[t ∈ [light_onset − 10, light_onset)] )
```

```
ΔAUC = ∫[light_lo → light_hi]  (signal(t) − baseline_mean)  dt  ≈  Σ trapezoids
```

Also saved as `auc_per_sec = ΔAUC / (light_hi − light_lo)` to allow comparison across cue types with different window lengths.

Light window boundaries (seconds, relative to cue onset):

Trial type	Light onset	Light offset
`Light`	0	30
`Conflict`	0	30
`Light-Tone`	0	30
`Tone-Light`	15	45

8.6 Total reward consumption

Reward volume is estimated from the duration of feeder activation:

```
feeder_seconds      = Σ  (Feeder active[t] > 0)   across all t in session
total_reward_uL     = feeder_seconds × FEEDER_RATE_UL_PER_SEC
total_reward_mL     = total_reward_uL / 1000
```

`FEEDER_RATE_UL_PER_SEC` (default: 16 µL/s) is set in `config.py`. Because the signal is fractional after resampling, a sample of 0.6 at a given second contributes 0.6 × 0 (it is below the 0 threshold in the `> 0` test) — however, since feeder states are typically sustained for multiple seconds, partial-second edge effects are negligible in practice.

One row per animal is written to `reward_summary.csv`. A bar chart (`reward_summary.svg`) shows total mL per animal.

8.7 Rewards during light trials

The same feeder-active counting is applied, but restricted to seconds that fall within a 30-second window after any detected Light cue onset:

```
light_seconds  = Uₜ  [light_onset_t, light_onset_t + 30)
feeder_in_light = Σ  (Feeder active[s] > 0)   for s ∈ light_seconds
reward_uL      = feeder_in_light × FEEDER_RATE_UL_PER_SEC
```

The light onset times are drawn from the auto-detected cue dictionary (`cue_dict["Light"]`), not hardcoded. An additional summary column `feeder_s_per_trial = feeder_in_light / n_light_trials` normalizes for sessions with different numbers of light trials.

8.8 Reward-Avoidance Index

For each cue type and each trial, the pipeline counts the seconds the animal spent in the reward zone versus on the safety platform:

```
R  = Σ  (In Reward[t] > 0.5)   for t ∈ [cue_onset, cue_onset + cue_duration)
P  = Σ  (In platform[t] > 0.5) for t ∈ [cue_onset, cue_onset + cue_duration)

RA_Index = (R − P) / (R + P)
```

The index ranges from **−1** (animal spent all cue time on the platform) to **+1** (animal spent all cue time in the reward zone). A value of 0 indicates equal time in both zones. The index is **NaN** when neither zone was occupied during the cue (denominator = 0), meaning the animal was elsewhere in the apparatus.

The index is first computed per trial, then per animal (summing seconds across all trials of a given type before computing the index), and finally averaged across animals within each treatment group for the bar chart. The group bar chart includes ± SEM error bars and a dashed zero line for reference.

8.9 Latency to platform

For each trial, latency is measured from **tone onset** in a 35-second search window. Tone onset equals trial start for all trial types except `Light-Tone`, where tone onset = trial start + 15 s (the light precedes the tone).

```
latency = time_of_first_platform_entry − tone_onset
```

Within the search window `[tone_onset, tone_onset + 35 s]`:

- If `In platform ⩾ 0.5` at the **first sample** of the window → `latency = 0` (animal was already on the platform at tone onset)
- If a 0→1 transition occurs at any later sample → `latency = time_of_that_sample − tone_onset`

- If no entry occurs → `latency = NaN` and `Never_On_Platform = True`

NaN latencies are excluded from group mean calculations but are reported in the raw CSV and used separately to compute the % **trials never reaching the platform** figure.

8.10 Shock outcome classification

Each tone-containing trial (`Tone`, `Conflict`, `Light-Tone`, `Tone-Light`) is classified by examining `In platform` at the **last 2 seconds of the tone period.**

The tone period is identified by walking forward from tone onset while `tone_cue == 1` in the downsampled binary data, collecting all tone-active sample indices. If fewer than 2 tone-active samples are found (edge effect), the window falls back to the fixed interval `[tone_onset, tone_onset + tone_duration]`. The final two indices of the resulting sequence are then examined:

`In platform` at second-to-last sample	`In platform` at last sample	Classification
1	1	**Avoided** — animal was on platform for the full shock window
0	1	**Escaped** — animal reached platform before tone offset
0	0	**Fully Shocked** — animal was off platform for the full shock window
NaN in either position	—	Trial excluded from all calculations

Per-mouse proportions are computed before group averaging:

```
avoided_pct (animal)       = 100 × (# Avoided trials)       / (# valid trials)
escaped_pct (animal)       = 100 × (# Escaped trials)       / (# valid trials)
fully_shocked_pct (animal) = 100 × (# Fully Shocked trials) / (# valid trials)
```

Group means are computed by averaging per-animal percentages, giving each animal equal weight regardless of how many trials it contributed. Two versions of stacked bar figures are produced:

- **POOLED** — percentages computed from raw trial counts pooled across all animals in a group (biased toward animals with more trials)

- **PERMOUSE** — per-animal percentages computed first, then averaged across animals (the statistically appropriate method; equal animal weighting)

> **Note on the original shock analysis approach:** Yarur's original `Shock_Analysis.ipynb` classified avoidance by detecting actual `New shocker active` TTL onset events and asking whether the animal was on the platform during the shock delivery window. The current pipeline instead classifies by the last 2 seconds of the *tone* period, which yields three outcome categories (Avoided / Escaped / Fully Shocked) based on cue-relative behavior rather than shock delivery itself. These are complementary measures capturing related but distinct aspects of avoidance performance.

8.11 Across-session AUC

The per-mouse `auc_baseline_subtracted` values (Section 8.5) saved by the nosepoke analysis are aggregated across sessions. For each cue type and each session, group means and SEM are computed across animals:

```
group_mean_AUC[session]  = mean( ΔAUC_j )   across animals j in the group
group_SEM_AUC[session]   = SD( ΔAUC_j ) / √n
```

These are plotted as line graphs with error bars across sessions, one line per treatment group and one panel per cue type.

---

9. Output Files and Folder Structure

All outputs are written inside `BehaviorData/Analysis/`. Nothing is written outside of it.

```
BehaviorData/
└── Analysis/
    ├── VC01/
    |   ├── time_on_platform/
    |   |   ├── *_combined_platform_summary.csv     ← long-format; one row per
animal × cue × timepoint
    |   |   ├── *_group_mean_platform.svg            ← group mean ± SEM traces,
one panel per cue
    |   |   ├── *_grid_Light.svg                     ← per-mouse trace grids, one
file per cue
    |   |   ├── *_grid_Conflict.svg
    |   |   └── *_platform_LIGHT_auc_per_mouse.csv  ← one row per animal × light-
containing cue
    |   |
    |   ├── nosepokes/
    |   |   ├── *_combined_nosepoke_summary.csv
    |   |   ├── *_group_mean_nosepoke.svg
    |   |   ├── *_grid_Light.svg
    |   |   ├── *_nosepoke_LIGHT_auc_per_mouse.csv  ← consumed by across-session
AUC analysis
    |   |   └── session_nosepoke_histogram.svg       ← 30 s binned whole-session
view, all animals
    |   |
    |   ├── rewards/
    |   |   ├── reward_consumption/
    |   |   |   ├── reward_summary.csv              ← one row per animal:
feeder_s, uL, mL
    |   |   |   └── reward_summary.svg              ← bar chart of total mL per
animal
    |   |   ├── rewards_in_light_trials/
    |   |   |   └── rewards_in_light_trials.csv     ← feeder counts gated to light
windows
    |   |   └── reward_avoidance_index/
    |   |       ├── reward_avoidance_index_per_trial.csv   ← one row per animal ×
cue × trial
    |   |       ├── reward_avoidance_index_summary.csv     ← one row per animal ×
cue (summed)
    |   |       └── reward_avoidance_index.svg             ← group mean ± SEM bar
chart
    |   |
    |   ├── latency/
    |   |   └── latency_summary.csv                 ← one row per animal × trial
    |   |
```

```
    |   └── shocks/
    |       ├── shock_trial_outcomes.csv            ← one row per classified
trial
    |       ├── shock_outcomes_by_trialtype.csv     ← per-animal counts by trial
type + total
    |       ├── stacked100_POOLED.svg
    |       ├── stacked100_PERMOUSE.svg
    |       ├── barplot_outcomes_by_trialtype.svg   ← mean bars + individual
animal points
    |       ├── cumulative_shocks_group_mean.svg
    |       └── cumulative_shocks_individuals.svg
    |
    ├── VC02/  ...
    ├── REW01/ ...
    |
    └── across_sessions/
        ├── latency/
        |   ├── latency_combined.csv
        |   ├── latency_across_sessions.svg         ← scatter + mean ± SEM per
session per cue
        |   └── never_on_platform_across_sessions.svg
        |
        ├── shocks/
        |   ├── per_mouse_outcome_percents.csv
        |   ├── treatment_mean_outcome_percents.csv
        |   ├── across_session_stacked100_POOLED.svg
        |   ├── across_session_stacked100_PERMOUSE.svg
        |   └── across_session_line_collapsed.svg   ← outcomes collapsed across
trial types
        |
        └── auc/
            ├── nosepoke_auc_long.csv               ← one row per animal ×
session × cue
            ├── nosepoke_auc_by_treatment.svg
            └── nosepoke_auc_by_sex.svg             ← only produced if sex is in
metadata
```

Combined summary CSV columns (platform and nosepoke)

Column	Description
`Mouse ID`	Full filename stem (e.g. `07-15-25_10b_VC01`)

Column	Description
`Cue`	Trial type (`Light`, `Tone`, `Conflict`, `Light-Tone`, `Tone-Light`)
`Time (s)`	Seconds relative to cue onset; ranges from −15 to `cue_duration + 30`
`Mean`	Trial-averaged behavioral fraction at this timepoint (0–1)
`SEM`	Standard error of the mean across trials for this animal

Reward summary CSV columns

Column	Description
`Mouse_ID`	Full filename stem
`Feeder_Active_s`	Total seconds of feeder activity across the session
`Total_Reward_uL`	`Feeder_Active_s × FEEDER_RATE_UL_PER_SEC`
`Total_Reward_mL`	`Total_Reward_uL / 1000`

Rewards in light trials CSV columns

Column	Description
`Mouse_ID`	Full filename stem
`N_light_trials`	Number of Light cue onsets in the session
`Feeder_Active_in_Light_s`	Feeder-active seconds falling within any light window
`Reward_in_Light_uL`	`Feeder_Active_in_Light_s × FEEDER_RATE_UL_PER_SEC`
`Reward_in_Light_mL`	`Reward_in_Light_uL / 1000`
`Feeder_s_per_trial`	`Feeder_Active_in_Light_s / N_light_trials`

Reward-Avoidance Index CSV columns

Column	Description
`Mouse_ID`	Full filename stem

Column	Description
`Cue_Type`	Trial type
`Trial_Index`	1-based index within this cue type (per-trial file only)
`Seconds_In_Reward`	Seconds with `In Reward > 0.5` during the cue window
`Seconds_In_Platform`	Seconds with `In platform > 0.5` during the cue window
`RA_Index`	`(Seconds_In_Reward − Seconds_In_Platform) / (Seconds_In_Reward + Seconds_In_Platform)`

Latency summary CSV columns

Column	Description
`Mouse_ID`	Full filename stem
`Behavior_ID`	Second `_`-delimited field from filename
`Trial_Type`	Cue type label
`Trial_Index`	1-based occurrence index for this cue type within the session
`Tone_Onset_s`	Absolute time (s) of tone onset in the recording
`Window_Length_s`	Search window length (default 35 s)
`Latency_s`	Seconds from tone onset to first platform entry; NaN if never reached
`Never_On_Platform`	True if animal did not enter the platform during the search window
`Session`	Parsed from filename (e.g. `VC01`)
`treatment_group`	Normalized label from metadata

Shock trial outcomes CSV columns

Column	Description
`Behavior_ID`	Lowercased second field from filename
`Treatment_Group`	Normalized label from metadata

Column	Description
`Trial_Type`	Tone-containing trial type
`Trial_Label`	e.g. `Conflict_03` (type + 1-based counter within that type)
`Trial_Index`	Chronological index across all tone trials in the session
`Trial_Start_Time`	Absolute trial onset time (s)
`Outcome`	`Avoided`, `Escaped`, or `Fully_Shocked`
`Session`	Added during across-session collection

10. Troubleshooting

`behavior_id 'XYZ' not in metadata` The behavior ID extracted from the filename (second `_`-delimited field, lowercased) was not found in `animals_metadata.xlsx`. Check that the two match exactly modulo case. The most reliable fix is to ensure the second field of your filenames matches the `behavior_id` column in your spreadsheet character-for-character.

`No downsampled/ folder found – skipping VC01` The session folder contains raw CSVs but `RUN_DOWNSAMPLE = False` was set on a previous run, so `downsampled/` was never created. Set `RUN_DOWNSAMPLE = True` and re-run, or manually place your downsampled files in a `downsampled/` subfolder inside the session folder.

`Could not extract cue_dict from first CSV – skipping VC01` `CUE_DICT_SOURCE = "first_csv"` and auto-detection failed. Either the `downsampled/` folder does not exist, contains no CSV files, or every file is missing the expected tone or light columns. Switch to `CUE_DICT_SOURCE = "config"` and fill in your known onset times to bypass auto-detection entirely, or check that your AnyMaze export includes `New speaker active` (or `Speaker Channel 1 active`) and `Cue light active`.

`Could not extract per-animal cue dicts – skipping VC01` `CUE_DICT_SOURCE = "per_animal"` and `extract_cue_dict()` returned an empty result for all files in `downsampled/`. Same column-name checks apply as above.

`No downsampled/ folder – skipping VC01` `CUE_DICT_SOURCE` is `"first_csv"` or `"per_animal"` and the `downsampled/` subfolder does not exist yet. Either set `RUN_DOWNSAMPLE = True` and re-run, or switch to `CUE_DICT_SOURCE = "config"` and provide onset times manually.

`No summaries produced for platform` **or** `nosepoke` No cue onset was found within 0.5 s of any sample in the animal's `Time (s)` column. This can happen if the cue dictionary was extracted from an animal with different trial timing than the one being analyzed. Confirm that all animals in a session ran on an identical trial schedule.

`'Feeder active' not found` The `Feeder active` column is missing from the raw CSV. Check your AnyMaze export settings and ensure the feeder zone is enabled. If your column is named differently (e.g. `Feeder Active` or `Feeder_active`), the simplest fix is to rename it in the CSV; alternatively, add a mapping for it in `load_behavior_fractional()` in `utils.py`.

`Missing columns {'In Reward'}` The Reward-Avoidance Index requires an `In Reward` AnyMaze zone column. If your apparatus does not have a distinct reward zone sensor, set `RUN_REWARDS = False` or accept that the R-A Index will be skipped for files missing that column (the reward consumption and rewards-in-light analyses will still run). The per-animal warning is printed but does not halt the rest of the pipeline.

`No Light cue times in cue_dict; skipping rewards-in-light-trials` The session had no `Light` trials detected (e.g. a VC session that began with Tone-only trials, or a protocol variant without Light cues). This is expected and safe to ignore if your session design does not include Light-only trials.

Figures appear but some panels are blank A blank panel means no data exists for that treatment group × cue combination after filtering. This is expected if not all groups received all cue types. It can also occur if all animals in a group returned `treatment_group = "Unknown"` due to a metadata mismatch and were excluded.

`No AUC records found in nosepokes/ subfolders` The across-session AUC analysis looks for `*_nosepoke_LIGHT_auc_per_mouse.csv` inside `Analysis/*/nosepokes/`. This file is only created when `RUN_NOSEPOKES = True` has been run successfully for that session. Check that the nosepoke within-session analysis completed without errors before running the across-session AUC step.

`No across-session latency / shock data found` The across-session collectors look for `latency_summary.csv` in `Analysis/*/latency/` and `shock_trial_outcomes.csv` in `Analysis/*/shocks/`. These are written by the within-session latency and shock analyses. Run those first, or check that their output folders are named exactly as expected.

Shock outcomes show only `Fully_Shocked` **for all trials** This usually means `In platform` is all zeros in the downsampled file. Check that AnyMaze exported the platform zone as a behavioral state column (not just an event) and that the column is named exactly `In platform`. The column mapping in `_load_downsampled()` inside `Variable_Conflict_Shocks.py` is explicit — any deviation in naming will cause it to be missed.

AnyMaze exported `Speaker Channel 1 active` **instead of** `New speaker active` Both are supported. The pipeline checks for both names when detecting the tone cue. If your column uses

a different name entirely, add it to the `col_map` dictionary inside `_load_downsampled()` in `Variable_Conflict_Shocks.py` and to the `_process()` function inside `extract_cue_dict()` in `utils.py`.

Treatment group appears as `Unknown` in all outputs The behavior ID was matched to metadata, but the `treatment_group` value in that row did not match any key in `TREATMENT_ALIASES` and was returned as-is. If the as-is label does not match a key in `TREATMENT_COLORS`, the animal will still be plotted (in the fallback color ● `#000000`), but if it is `"Unknown"` exactly, it will be excluded from group figures. Check the spelling of your treatment labels in the spreadsheet and ensure they are covered by `TREATMENT_ALIASES`.

Import error on `platform` Python has a built-in standard library module called `platform`. Do not rename `Time_on_Platform.py` to `platform.py` — imports will silently resolve to the wrong module.